

Zain Catering Services - Project Documentation

Zain Catering Services — Business Automation System

Complete Project Documentation

Stack: n8n (self-hosted, Docker) + PostgreSQL + NocoDB + Gotenberg + Cloudflare Tunnel **Client:** Zain Catering Services (Biryani, Qorma, Naan, Qulfa + tents, chairs, sound system rentals) **Status:** Built, validated, ready for deployment and live testing

Table of Contents

- [Project Background & Original Ask](#)
 - [Discovery Findings \(What We Learned About the Business\)](#)
 - [Architecture Overview](#)
 - [Data Model](#)
 - [Project Folder Structure](#)
 - [Infrastructure Setup \(Docker Compose\)](#)
 - [Database Setup \(Schema + Seed Data\)](#)
 - [Workflow-by-Workflow, Node-by-Node Documentation](#)
 - [The Combined Workflow](#)
 - [Invoice Template](#)
 - [Installation & Running Instructions](#)
 - [Testing Checklist](#)
 - [Known Simplifications & Assumptions](#)
 - [Future Expansion Roadmap](#)
-

1. Project Background & Original Ask

Zain runs a catering business in Pakistan — biryani, qorma, naan, qulfa, plus full event arrangement (tents, chairs, sound system, crockery). The original proposal asked for an n8n-powered automation to handle:

- Order intake (from phone calls, either client-filled or Zain-filled)
- Client record management
- Inventory tracking (both food and equipment)
- Billing sheet and PDF invoice generation
- Payment due reminders, including tracking clients who promise to pay later
- Customer feedback collection
- Client re-engagement for inactive customers

The system needed to be practical for a real phone-call-driven business — not a generic e-commerce flow — and had to run on a locally hosted n8n instance via Docker.

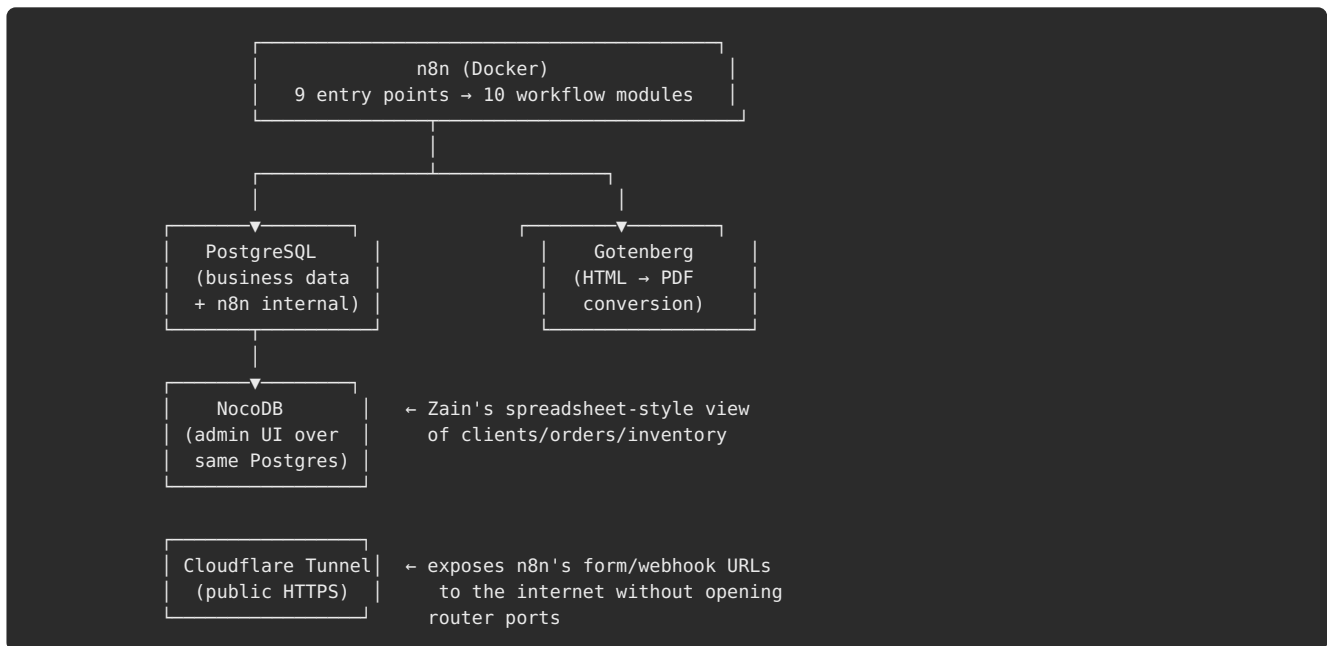
2. Discovery Findings

Before building, we gathered the specifics that shape the whole system:

Area	Finding	Design Impact
Business type	Catering (food + equipment rental), B2C mostly, 4-5 orders/week	Two distinct inventory models needed (see below)
Order flow	Phone call → Zain asks name, menu choice, guest count, equipment, date, phone number	Order form mirrors this exact conversation, no price field
Pricing	Negotiated <i>after</i> the form is submitted, over a follow-up call	Order lifecycle has a new →

		<code>pending_pricing</code> / <code>confirmed</code> split
Historical data	None — fresh start	All client/order/invoice tables start empty; only menu/equipment seeded
Low stock threshold	20% of baseline for both food and equipment	<code>low_stock_threshold_pct</code> column, defaulted to 20
Payments	Cash, JazzCash, EasyPaisa, bank — and clients who promise to pay later	Dedicated <code>is_promise</code> / <code>promised_date</code> / <code>promise_status</code> fields on <code>payments</code>
Equipment conflicts	Soft warning only, Zain decides (may rent extra externally)	IF node warns via email but always proceeds with the booking

3. Architecture Overview



Why this stack:

- **Postgres, not Sheets/Airtable** — the data has real relationships (orders → menu items → invoices → payments) that need joins and constraints, not spreadsheet rows.
- **n8n's native Form Trigger, not Typeform/Jotform** — one less external service, no extra monthly cost, fully self-hosted.
- **Gotenberg, not a paid PDF API** — free, self-hosted, full control of the invoice's HTML/CSS.
- **NocoDB** — gives Zain a day-to-day spreadsheet-style interface without needing to open n8n or write SQL.
- **Cloudflare Tunnel** — the only piece that *must* reach the public internet (client order form, feedback links). Avoids port-forwarding and works even from a home PC.

4. Data Model

Eleven tables, all in the `public` schema of the single Postgres database (n8n's own workflow data lives in a separate `n8n_internal` schema in the same instance):

Table	Purpose
<code>clients</code>	Name, phone (unique key), WhatsApp number, email, address, <code>last_order_at</code>
<code>menu_items</code>	Food items — cost, consumption_per_person , current stock, baseline, threshold
<code>equipment_items</code>	Rentals — total owned quantity, category
<code>orders</code>	One row per event — ref, event date, guest count, status, source
<code>order_menu_items</code>	Which food items + how many guests each order needs
<code>order_equipment</code>	Which equipment + quantity each order needs

equipment_bookings	Date-scoped equipment allocation — this is what makes soft-warning conflict checks possible
invoices	One per order — total, paid amount, generated column balance
payments	Real payments AND promise-to-pay entries (is_promise , promised_date , promise_status)
feedback	Rating + comment per order
reminder_log	Audit trail so reminders aren't sent twice

The single most important design decision in this schema: food and equipment are modeled completely differently.

- **Food** (**menu_items**) is a simple consumable — once used, it's gone. Deducted permanently: **stock -= guest_count × consumption_per_person** .
- **Equipment** (**equipment_items** + **equipment_bookings**) is reusable. It isn't deducted at all — it's *booked against a date* and freed up again the day after the event. Treating chairs like biryani would show Zain as permanently "out of chairs" after his very first event.

Two Postgres sequences (**order_ref_seq** , **invoice_number_seq**) generate clean, fresh-start reference numbers: **ZC-2026-0001** , **ZC-INV-2026-0001** .

5. Project Folder Structure

```
zain-catering-automation/
├── docker-compose.yml          # n8n + Postgres + NocoDB + Gotenberg + Cloudflare Tunnel
├── .env.example                # copy to .env, fill in real secrets
├── README.md                   # setup guide + deployment notes
├── postgres-init/
│   ├── 01_schema.sql           # all 11 tables, constraints, indexes, sequences
│   └── 02_seed_data.sql        # demo menu items + equipment quantities
├── invoice-template/
│   └── invoice_template.html    # Zain Catering Services branded invoice layout
├── workflows/                  # the 10 modules as separate importable files
│   ├── 01_order_intake.json
│   ├── 02_order_confirm_and_price.json
│   ├── 03_invoice_pdf_generation.json
│   ├── 04_payment_logging.json
│   ├── 05_daily_reminder_sweep.json
│   ├── 06_post_event_completion.json
│   ├── 07_feedback_request.json
│   ├── 08_feedback_intake.json
│   ├── 09_reengagement_sweep.json
│   └── 10_global_error_handler.json
└── zain_catering_COMBINED_WORKFLOW.json # all 10 modules merged into ONE workflow
                                         # (88 nodes, 9 entry points, one import)
```

On the host machine, Docker also creates (and persists data in) three volume folders alongside **docker-compose.yml** : **n8n-data/** , **nocodb-data/** , **postgres-data/** . These are created automatically the first time you run **docker compose up** .

6. Infrastructure Setup

docker-compose.yml defines five services on one bridge network (**zain_net**):

Service	Image	Port	Role
postgres	postgres:16-alpine	5432	Single source of truth for all business data + n8n's own workflow storage
n8n	docker.n8n.io/n8nio/n8n:latest	5678	The automation engine — every workflow in this project runs here
nocodb	nocodb/nocodb:latest	8080	Spreadsheet UI over the same Postgres database, for Zain's daily use
gotenberg	gotenberg/gotenberg:8	3000	Headless Chromium — converts invoice HTML to PDF
cloudflared	cloudflare/cloudflared:latest	—	Tunnels n8n's webhook/form URLs to the public internet

Key details:

- n8n is configured with `DB_TYPE=postgresdb` pointing at the *same* Postgres container as the business data, but a separate schema (`n8n_internal`) — one database, one backup, no collision with business tables.
- n8n's editor UI is protected with HTTP Basic Auth (`N8N_BASIC_AUTH_USER` / `PASSWORD` from `.env`).
- Timezone is fixed to `Asia/Karachi` so all scheduled workflows (9am, 6am, Monday 9am) fire at the correct local time.
- `cloudflared` requires a one-time manual setup outside Docker (`cloudflared tunnel login` / `create` / `token`) — the resulting token goes into `.env` as `CLOUDFLARE_TUNNEL_TOKEN` .

7. Database Setup

Two SQL files run automatically on the **first** container start (Postgres's `docker-entrypoint-initdb.d` mechanism — they will *not* re-run on subsequent restarts unless the `postgres-data` volume is wiped):

`01_schema.sql` creates:

- The `n8n_internal` schema (empty, n8n populates it itself on first launch)
- All 11 business tables with foreign keys, `CHECK` constraints (e.g. `orders.status` can only be `new` / `pending_pricing` / `confirmed` / `completed` / `cancelled`), and indexes on the columns queried most often (`event_date` , `status` , `promised_date`)
- `invoices.balance` as a **generated column** (`total - paid_amount`) — always correct, never needs a workflow step to maintain it
- Two sequences for order/invoice numbering

`02_seed_data.sql` inserts:

- 4 menu items: Chicken Biryani, Mutton Qorma, Plain Naan, Qulfa (Kulfi) — each with a demo `consumption_per_person` (e.g. 0.35 kg biryani per guest) and 50/40/500/100 starting stock
- 10 equipment items: Standard Chair, Round Dining Table, Large Tent, Small Tent, Sound System Basic/Premium, Plates, Glasses, Spoons, Water Jugs

Important assumption: on a freshly created database, Postgres's `SERIAL` columns assign IDs in insertion order — so menu items are guaranteed to be IDs 1-4 and equipment items 1-10 in the exact order listed above. The Order Intake workflow's mapping code hardcodes these IDs. If you ever re-seed or reorder these inserts, that mapping must be updated to match.

Everything else — `clients` , `orders` , `invoices` , `payments` , `feedback` — starts completely empty, per the "fresh start, no historical data" instruction.

8. Workflow-by-Workflow, Node-by-Node Documentation

Module 01 — Order Intake

Entry points: two Form Triggers. Purpose: capture an order however it comes in, and get it into the database identically either way.

Node	Type	What it does
Client Order Form	Form Trigger	Public form (path <code>/client-order-form</code>) — the link Zain texts a client after a call. Fields: name, phone, WhatsApp, email, event date, venue, guest count, per-item quantity fields for each menu item and equipment type, a Sound System dropdown, and notes.
Internal Order Form	Form Trigger	Identical field set, different path (<code>/internal-order-form</code>) — Zain fills this himself while still on the phone.
Tag Source: Client Form / Tag Source: Internal Form	Code	Stamps a <code>__source</code> field (<code>client_form</code> / <code>internal_form</code>) onto the incoming data so the order record knows how it arrived. Both branches feed into the same next node.
Normalize Order Data	Code	The real translation layer: reads the raw form field labels (e.g. <code>"Chicken Biryani - servings needed..."</code>) and reshapes them into a clean object — <code>menu_items[]</code> and <code>equipment[]</code> arrays (filtering out any item left at 0), plus the core order fields. This is where the hardcoded menu/equipment ID mapping lives.
Upsert Client	Postgres	<code>INSERT ... ON CONFLICT (phone) DO UPDATE</code> — matches existing clients by phone number, updates name and (if provided) email, without creating duplicates for a repeat caller.
Get Next Order Ref	Postgres	Pulls the next value from <code>order_ref_seq</code> .
Format Order	Code	Builds the human-readable ref (<code>ZC-2026-0001</code>) and re-attaches the client ID and original order data for

Ref + Merge Context		the next steps.
Insert Order	Postgres	Creates the order row with <code>status = 'new'</code> . This is the fan-out point — four things happen in parallel from here.
Update Client Last Order Date	Postgres	Sets <code>clients.last_order_at = now()</code> — this is what feeds the re-engagement sweep later.
Split Menu Items	Code	Takes the <code>menu_items</code> array and turns it into one item per row, with the real <code>order_id</code> now attached (referenced via <code>\$('#Insert Order')</code> , not passed through the graph, to avoid a sequencing race).
Insert Order Menu Items	Postgres	Runs once per menu line (n8n's default per-item execution), inserting into <code>order_menu_items</code> .
Split Equipment Items	Code	Same pattern as above, for equipment.
Insert Order Equipment	Postgres	Runs once per equipment line.
Notify Zain - New Order	Email	Sends Zain the order ref, client name/phone, event date, guest count, venue, and notes — the cue to call the client and negotiate pricing.

Module 02 — Order Confirm and Price

Entry point: one Form Trigger. This is the orchestrator — the single biggest workflow, doing pricing, stock deduction, equipment booking with conflict warning, and kicking off invoice generation, all in parallel branches off one node.

Node	Type	What it does
Confirm Order Form	Form Trigger	Zain enters: order ref, agreed total price, advance paid now, advance payment method.
Normalize Input	Code	Cleans/types the form values (numbers as numbers, method lowercased).
Find Order By Ref	Postgres	Looks up the order by its ref to get <code>id</code> , <code>client_id</code> , <code>event_date</code> , <code>guest_count</code> .
Update Order Status: Confirmed	Postgres	Sets <code>status = 'confirmed'</code> , <code>confirmed_at = now()</code> . Fan-out point — three branches run from here: food, equipment, invoice.
[Food branch] Get Order Menu Items	Postgres	Joins <code>order_menu_items</code> with <code>menu_items</code> to get each item's consumption rate and current stock.
Calculate Deduction	Code	<code>deduction = guest_count_applicable × consumption_per_person</code> ; computes <code>newStock</code> .
Deduct Menu Item Stock	Postgres	<code>UPDATE menu_items SET stock_qty_raw_material = newStock WHERE id = ...</code>
IF Low Stock	IF	Checks <code>(stock / baseline) × 100 <= threshold_pct</code> (the 20% rule).
Send Low Stock Alert	Email	Fires only when the IF is true — tells Zain exactly which item and current level.
[Equipment branch] Get Order Equipment	Postgres	Joins <code>order_equipment</code> with <code>equipment_items</code> for requested quantities and totals owned.
Check Equipment Availability	Postgres	The soft-warning query: for the order's event date, sums everything already <code>booked</code> for that equipment and date, subtracts from <code>total_qty</code> to get <code>available</code> .
IF Availability Conflict	IF	<code>available < qty_requested</code> .
Send Equipment Conflict Warning	Email	Fires on conflict — explicitly labeled a soft warning ; both IF branches (warned or not) proceed to the same next step.
Insert Equipment Booking	Postgres	Always runs regardless of the warning — inserts into <code>equipment_bookings</code> with <code>status = 'booked'</code> . This is the deliberate "warn but don't block" behavior you approved.
[Invoice branch] Get Next Invoice Number	Postgres	Pulls from <code>invoice_number_seq</code> .
Format Invoice Number	Code	Builds <code>ZC-INV-2026-0001</code> ; computes invoice status (<code>paid</code> / <code>partial</code> / <code>unpaid</code>) from advance vs. total.
Insert Invoice	Postgres	Creates the invoice row.
IF Advance Paid	IF	Checks if <code>advance_paid > 0</code> .

Insert Advance Payment	Postgres	Only if true — logs the advance as a real (non-promise) payment.
Generate Invoice PDF (sub-workflow)	Execute Workflow	Calls Module 03 to actually build and send the PDF. <i>(In the combined workflow, this node is removed and replaced with a direct connection — see Section 9.)</i>
Notify Zain - Order Confirmed	Email	Confirms the price and that the invoice is being generated.

Module 03 — Invoice PDF Generation

Entry point: Execute Workflow Trigger (called by Module 02). Sequential by design — see the note on why it's not fanned out.

Node	Type	What it does
When Called by Another Workflow	Execute Workflow Trigger	Accepts <code>order_id</code> .
Get Invoice + Order + Client	Postgres	One joined query pulling everything needed for the invoice header: order ref, event date, guest count, venue, client name/phone/email, and the invoice's own totals.
Get Order Menu Items (Invoice)	Postgres	Menu line items for the invoice body. <i>(Renamed from "Get Order Menu Items" to avoid a name collision with Module 02 once merged — see Section 9.)</i>
Get Order Equipment (Invoice)	Postgres	Equipment line items. Chained sequentially after the menu query rather than run in parallel — this was a deliberate fix: fanning three lookups into one downstream node would have caused it to fire multiple times and send the invoice three times over.
Build Invoice HTML	Code	Loads the invoice template (pasted inline in production — see Section 10) and does token replacement: invoice number, dates, client info, menu/equipment rows built as HTML table rows, totals.
Convert HTML String to Binary	Code	Wraps the HTML string as an actual binary file (<code>index.html</code>) using <code>Buffer.from()</code> and n8n's <code>prepareBinaryData</code> helper — Gotenberg needs a real file upload, not a JSON string.
Convert HTML to PDF (Gotenberg)	HTTP Request	POSTs the HTML file to <code>http://gotenberg:3000/forms/chromium/convert/html</code> , gets the rendered PDF back as binary.
Upload Invoice PDF to Drive	Google Drive	Uploads the PDF, named after the invoice number, returning a <code>webViewLink</code> .
Update Invoice PDF URL	Postgres	Saves that link into <code>invoices.pdf_url</code> .
IF Client Has Email	IF	Guards the final step — many Pakistani catering clients won't have given an email.
Email Invoice to Client	Email	Sends the Drive link (not a raw attachment — more reliable than trying to pass binary data through several intervening nodes) with a short thank-you note.

Module 04 — Payment Logging

Entry point: Form Trigger. This directly solves the stated pain point — Zain no longer has to remember who owes what and when.

Node	Type	What it does
Log Payment Form	Form Trigger	Fields: invoice number, entry type (Payment Received Now / Client Promised to Pay Later), amount, method, promised date.
Normalize Input	Code	Cleans values; sets <code>is_promise</code> boolean from the entry type dropdown.
Find Invoice	Postgres	Looks up the invoice by number.

IF Is Promise	IF	Branches based on entry type.
Insert Promise Record	Postgres	If a promise: inserts into <code>payments</code> with <code>is_promise = true</code> , <code>promised_amount</code> , <code>promised_date</code> , <code>promise_status = 'pending'</code> — no money changes hands yet , this just logs the commitment.
Insert Actual Payment	Postgres	If real money: inserts a normal payment row with <code>paid_at = now()</code> .
Update Invoice Paid Amount	Postgres	Only runs on the real-payment path — increments <code>invoices.paid_amount</code> and recalculates <code>status</code> (<code>unpaid</code> / <code>partial</code> / <code>paid</code>) via a <code>CASE</code> expression.
Confirm Logged	Email	Simple confirmation back to Zain of what was recorded.

Module 05 — Daily Reminder Sweep

Entry point: Schedule Trigger, 9am daily. Three independent branches, each following the same pattern: query → IF rows exist → format → email.

Node	Type	What it does
Every Day 9am	Schedule Trigger	Cron <code>0 9 * * *</code> , <code>Asia/Karachi</code> timezone (set at the container level).
Get Promises Due Today	Postgres	<code>WHERE is_promise = true AND promise_status = 'pending' AND promised_date = CURRENT_DATE</code> — joined with client/invoice info.
IF Promises Exist	IF	Skips the email entirely if there's nothing due.
Format Promises Message	Code	Builds a plain-text list: client, phone, amount, invoice number.
Email Promises Due	Email	Sends the digest.
Get Overdue Balances	Postgres	<code>WHERE balance > 0 AND event_date < (CURRENT_DATE - 2 days)</code> — events that already happened but aren't fully paid.
IF Overdue Exist → Format Overdue Message → Email Overdue Balances	Same pattern as above.	
Get Upcoming Unpaid Events	Postgres	<code>WHERE balance > 0 AND event_date BETWEEN today AND today+2</code> — a heads-up before the event even happens.
IF Upcoming Unpaid Exist → Format Upcoming Message → Email Upcoming Unpaid	Same pattern.	

Module 06 — Post Event Completion

Entry point: Schedule Trigger, 6am daily.

Node	Type	What it does
Every Day 6am	Schedule Trigger	Cron <code>0 6 * * *</code> .
Mark Equipment Returned	Postgres	<code>UPDATE equipment_bookings SET status = 'returned' WHERE event_date < CURRENT_DATE AND status = 'booked'</code> — this is what frees up chairs/tents for future dates, closing the loop the conflict-check query depends on.
Mark Orders Completed	Postgres	<code>UPDATE orders SET status = 'completed' WHERE event_date < CURRENT_DATE AND status = 'confirmed'</code> , returning <code>id</code> , <code>client_id</code> per row.
Trigger Feedback Request (sub-workflow, per order)	Execute Workflow	Calls Module 07 once per just-completed order. (Removed and directly wired in the combined workflow — Section 9.)

Module 07 — Feedback Request

Entry point: Execute Workflow Trigger (called by Module 06).

Node	Type	What it does
When Called by Another Workflow	Execute Workflow Trigger	Accepts <code>id</code> (order id) and <code>client_id</code> .
Wait 12 Hours	Wait	A buffer so the feedback ask doesn't land immediately at 6am.
Get Client + Order Details	Postgres	Client name/email/phone and order ref.
IF Has Email	IF	Same email-optional guard as Module 03.
Send Feedback Request	Email	Includes a link to the feedback form pre-filled with <code>order_id</code> and <code>client_id</code> as query parameters.
Log Reminder Sent	Postgres	Writes to <code>reminder_log</code> (<code>type = 'feedback_request'</code>) so there's an audit trail.

Module 08 — Feedback Intake

Entry point: Form Trigger.

Node	Type	What it does
Feedback Form	Form Trigger	Order ID and Client ID (pre-filled via the emailed link's query string), a 1-5 rating dropdown, and a comments field.
Normalize Feedback Input	Code	Types the IDs and rating as numbers.
Insert Feedback	Postgres	Writes the row to <code>feedback</code> .
IF Rating Is Low	IF	<code>rating <= 2</code> .
Get Client Details	Postgres	Only on the low-rating path — pulls name/phone for the alert.
Send Low Rating Alert	Email	Flags Zain for a personal follow-up call — the service-recovery step.

Module 09 — Reengagement Sweep

Entry point: Schedule Trigger, Monday 9am weekly.

Node	Type	What it does
Every Monday 9am	Schedule Trigger	Cron <code>0 9 * * 1</code> .
Get Inactive Clients	Postgres	<code>WHERE last_order_at < (CURRENT_DATE - 45 days) OR (last_order_at IS NULL AND created_at < ...)</code> — catches both lapsed repeat clients and one-time clients from the early days who never came back.
IF Inactive Clients Exist	IF	Skips the email if the list is empty.
Format Reengagement Message	Code	Lists each client's name, phone, and last-order date (or "never ordered").
Email Reengagement List	Email	Sent to Zain as a prompt to personally reach out.

Module 10 — Global Error Handler

Entry point: Error Trigger (n8n's built-in mechanism — fires automatically when any workflow that names this one as its "Error Workflow" fails).

--	--	--

Node	Type	What it does
Error Trigger	Error Trigger	Receives the failed workflow's name, the node it failed at, and the error message.
Format Error Details	Code	Builds a readable plain-text summary.
Alert Developer	Email	Sent to the developer/maintainer (not Zain) — these are technical failures, not business events.

9. The Combined Workflow

`zain_catering_COMBINED_WORKFLOW.json` merges all 10 modules above into a single n8n workflow — **88 nodes, 9 independent entry points** (4 Form Triggers, 3 Schedule Triggers, 1 Execute Workflow Trigger... actually reduced to 0 Execute Workflow Triggers, since both cross-module calls were rewired — see below — and 1 Error Trigger).

Two changes were made specifically for this merge, beyond just concatenating node lists:

1. Name collisions resolved. Three node names existed in more than one module (`Get Order Menu Items` , `Get Order Equipment` appeared in both Module 02 and Module 03; `Normalize Input` appeared in both Module 02 and Module 04). Since n8n requires unique node names within one workflow, the Module 03 and Module 04 copies were renamed (`... (Invoice)` , `Normalize Payment Input`) and every expression elsewhere in those modules that referenced the old name (e.g. `$('Get Order Menu Items')`) was updated to match.

2. Sub-workflow calls replaced with direct connections. The individual modules use `Execute Workflow` / `Execute Workflow Trigger` node pairs to call Module 03 from Module 02, and Module 07 from Module 06. In the combined file, both `Execute Workflow` nodes and both `Execute Workflow Trigger` nodes were removed entirely, and the upstream node's output was wired directly into the downstream module's first real node instead. Expressions that referenced the (now-deleted) trigger node — e.g. `$('When Called by Another Workflow').first().json.order_id` — were rewritten to pull the same data from the actual upstream node instead (`$('Insert Invoice').first().json.order_id` , `$('Mark Orders Completed').first().json.id`).

This was done with a Python script rather than by hand, specifically to eliminate the risk of a missed reference or a silent duplicate name. Before writing the final file, the script verified programmatically:

- No duplicate node names anywhere in the merged 88-node set
- No duplicate node IDs
- Every connection's source node exists in the node list
- Every connection's target node exists in the node list

All four checks passed. This is the version recommended for import — one file, one click, already wired end to end, with no `WORKFLOW_ID` placeholder to go back and fix afterward.

10. Invoice Template

`invoice-template/invoice_template.html` is a standalone, styled HTML file with `{{token}}` placeholders (`{{invoice_number}}` , `{{client_name}}` , `{{menu_rows}}` , `{{balance}}` , etc.). It's kept as a separate file for readability, but Module 03's "Build Invoice HTML" code node needs the literal HTML string inline (or supplied via an `INVOICE_HTML_TEMPLATE` environment variable) since n8n Code nodes can't read arbitrary files off disk at runtime. This is a one-time copy-paste step during setup, documented in the README.

The template includes: Zain Catering Services branding header, invoice number/order ref/dates, client billing info, guest count, a menu table, an equipment table, and a totals block (total / paid / balance due) styled to stand out.

11. Installation & Running Instructions

Step 1 — Prepare environment

```
cd zain-catering-automation
cp .env.example .env
# edit .env: set real Postgres password, n8n admin password, NocoDB JWT secret
```

Step 2 — (Recommended) Set up Cloudflare Tunnel so client-facing forms work from any phone:

```
cloudflared tunnel login
cloudflared tunnel create zain-n8n
cloudflared tunnel token zain-n8n # paste result into .env as CLOUDFLARE_TUNNEL_TOKEN
```

Point a DNS record at the tunnel via the Cloudflare dashboard, then update `WEBHOOK_URL`, `N8N_HOST`, `N8N_PROTOCOL` in `.env` to match your real domain.

Step 3 — Launch the stack

```
docker compose up -d
```

This starts Postgres (running `01_schema.sql` and `02_seed_data.sql` automatically on first boot), n8n, NocoDB, Gotenberg, and the tunnel.

Step 4 — Open n8n at `http://localhost:5678` (or your tunnel domain), log in with the Basic Auth credentials from `.env`.

Step 5 — Import the workflow

- In n8n: **Workflows** → **Import from File** → select `zain_catering_COMBINED_WORKFLOW.json`.

Step 6 — Create credentials (n8n will prompt for these on nodes that need them):

- **Zain Catering Postgres** — Postgres credential, host `postgres`, port `5432`, database/user/password matching `.env`
- **Zain Catering SMTP** — for all outgoing email nodes
- **Zain Catering Google Drive** — OAuth2, for the invoice upload step

Step 7 — Paste the invoice template into the "Build Invoice HTML" node's placeholder (`PASTE_INVOICE_TEMPLATE_HTML_HERE`), replacing it with the full contents of `invoice-template/invoice_template.html`.

Step 8 — Fix two remaining placeholders:

- The Google Drive node's `folderId` (currently "root" — point it at a real shared folder)
- The `YOUR_N8N_DOMAIN` placeholder in the feedback request email, once your tunnel domain is live

Step 9 — Activate the workflow (toggle switch, top right of the n8n canvas).

Step 10 — Open NocoDB at `http://localhost:8080`, connect it to the same Postgres instance, and let it auto-detect the existing tables — this becomes Zain's day-to-day spreadsheet view.

12. Testing Checklist

Before handing off to Zain, walk through this sequence once with test data:

1. Submit the **Internal Order Form** with a made-up client, a couple of menu items, and one piece of equipment.
2. Confirm the order appears in `orders` (via NocoDB) with `status = 'new'`, and that Zain receives the "New Order" email.
3. Submit the **Confirm Order Form** with that order's ref and a test price.
4. Verify: `menu_items` stock decreased correctly, an `equipment_bookings` row was created, an `invoices` row exists, and (if Drive/SMTP credentials are set up) a PDF appears in Drive and an email arrives.
5. Submit the **Log Payment Form** once as a real payment and once as a "promised to pay later" — check both update the right tables.
6. Manually execute the **Daily Reminder Sweep** workflow (right-click → Execute Workflow in the n8n editor) rather than waiting for 9am, and confirm the digest email looks right.
7. Manually execute **Post Event Completion** against a test order with a past `event_date`, and confirm it flows through to a feedback request email.
8. Submit the **Feedback Form** with a low rating (1 or 2) and confirm Zain gets the low-rating alert.
9. Manually execute the **Reengagement Sweep** and check the inactive-client list looks correct.
10. Deliberately break something (e.g. a bad Postgres query) to confirm the **Error Trigger** fires and the developer alert arrives.

13. Known Simplifications & Assumptions

These are deliberate, documented shortcuts — not bugs — made to ship a working v1 without blocking on things outside this project's control:

- **Menu/equipment IDs are hardcoded** in Module 01's mapping code, assuming the database was seeded exactly as shipped. Reordering or adding items later means updating that mapping.
- **Consumption-per-person rates are estimates**, not Zain's real historical data (none existed) — correct via NocoDB once real events provide real numbers.
- **Client email is optional**. Most Pakistani catering clients communicate by phone/WhatsApp, not email — invoice and feedback emails are skipped gracefully (via an IF guard) rather than failing when no email is on file. WhatsApp Business API is the natural phase-2 upgrade once Meta's approval process is complete, and would slot in by swapping the `emailSend` nodes without touching anything upstream.
- **Feedback form's Order ID / Client ID fields are visible text inputs**, not truly hidden fields, since not all n8n versions

support query-param-to-hidden-field binding identically. They're pre-filled via the emailed link's URL, so in practice clients won't need to touch them — but this is a spot worth hardening (e.g. a signed token) if feedback data integrity becomes a concern later.

- **45-day re-engagement threshold** and **20% low-stock threshold** are sensible starting defaults, not numbers Zain specified from data — both are simple to tune once real order patterns exist.
- **Invoice delivery uses a Drive link in the email body**, not a raw PDF attachment — this was a deliberate reliability choice, since passing binary data through several intervening nodes (Drive upload → Postgres update → email) is fragile in n8n; a link is simpler and just as usable for the client.

14. Future Expansion Roadmap

Once the core automation has run for a few weeks on real data:

- **WhatsApp Business API integration** (pending Meta approval) — replaces email as the primary client-facing channel
- **AI-powered demand prediction** for food/equipment based on accumulated order history
- **Sentiment analysis on feedback comments**, not just the numeric rating
- **A basic reporting dashboard** (NocoDB views, or a small custom front-end) summarizing weekly orders, revenue, and outstanding balances at a glance
- **Recurring/standing order support**, if Zain confirms some clients order the same thing on a schedule

None of these require changes to the current data model — they build on top of what's already in place.